UNIVERSITY OF VOCATIONAL TECHNOLOGY

IT304082 – Database Implementation

Final Assignment

Project Title: UOVT Student Management System

Team Members:

- **G. B. D. Anuradha** – SIS24B215
- **L. B. Charith Jeewan** – SIS24B236
- **W. I. L. Withana** – SIS24B238
- **H. K. G. V. L. Koralage** – SIS24B213
- **B. W. S. S. Nawarathna** – SIS24B239

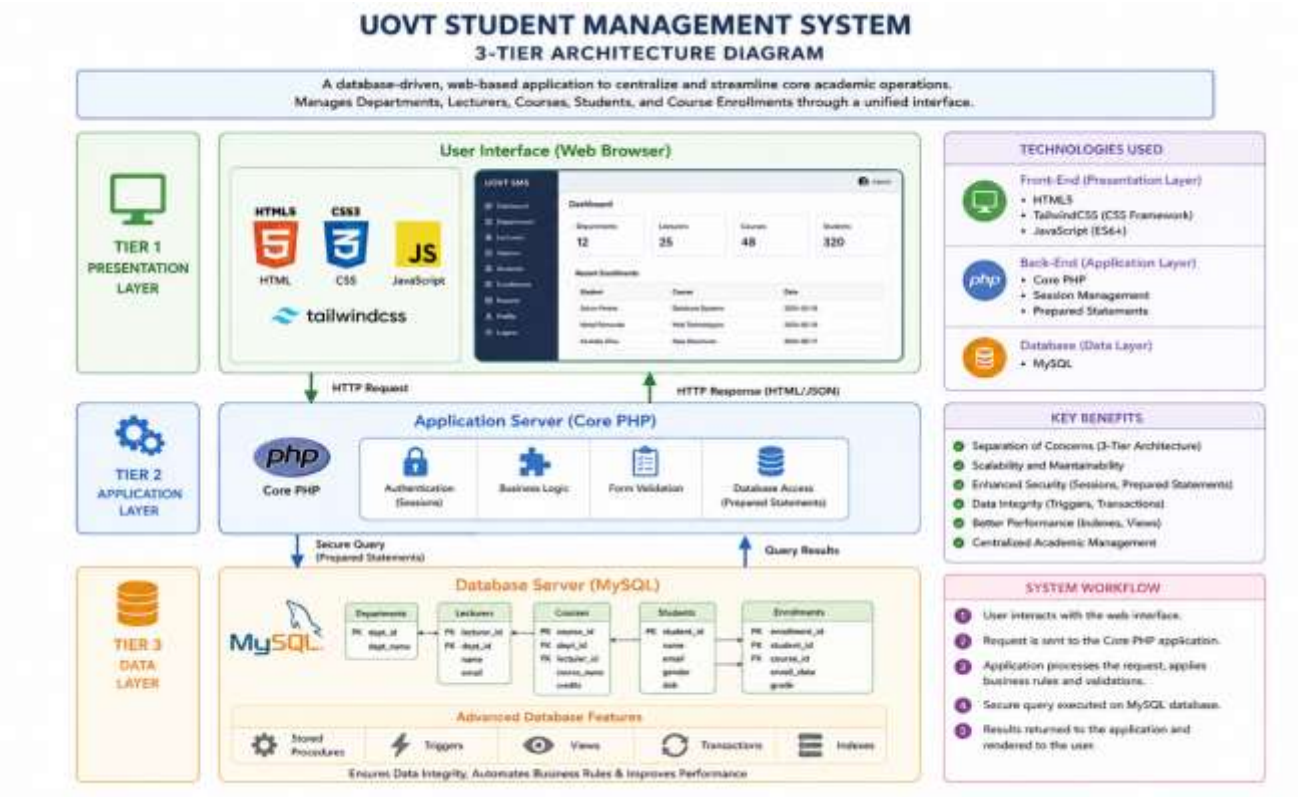
Date of Submission: May 6, 2026

1. Introduction

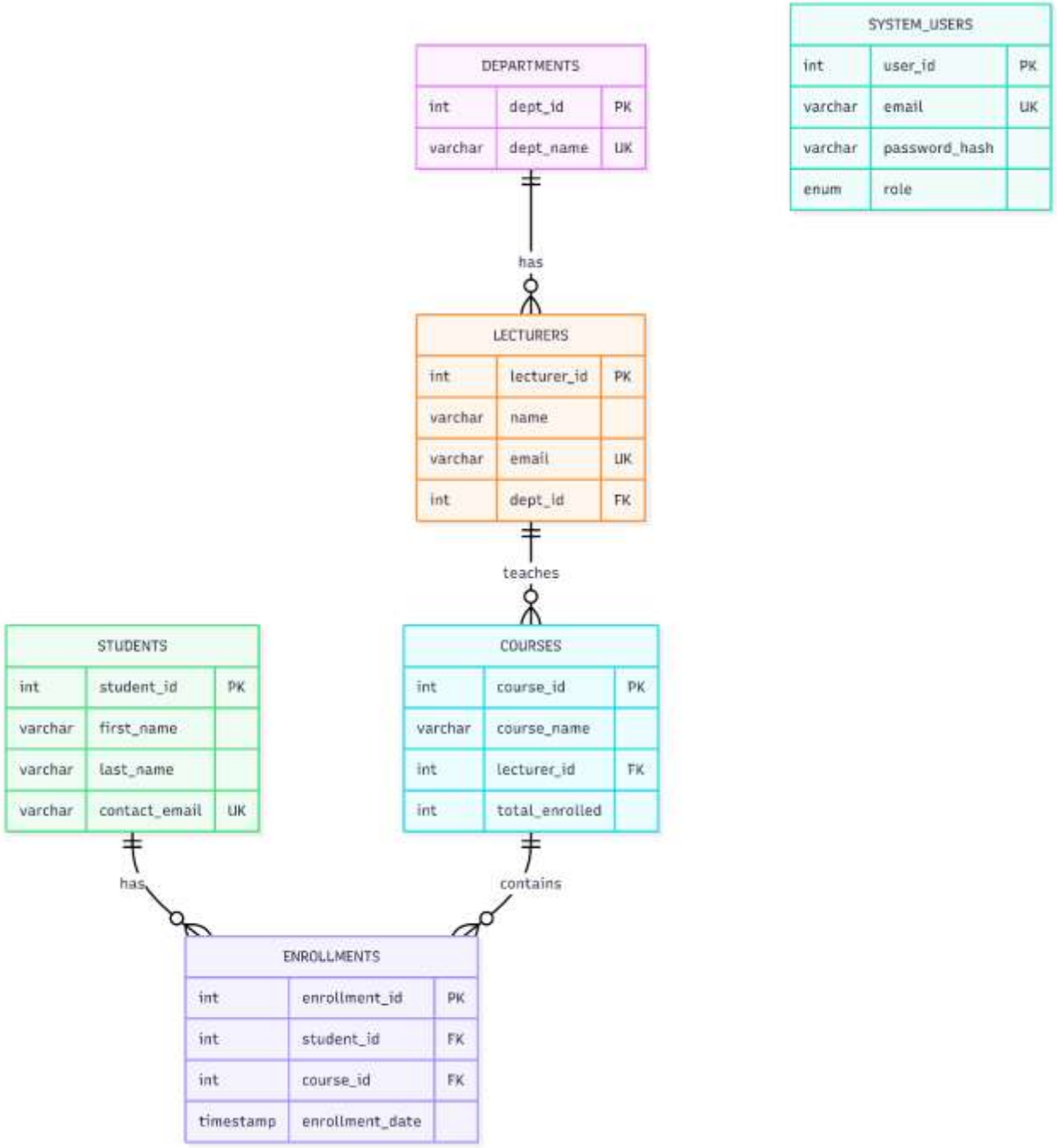
The **UOVT Student Management System** is a database-driven, web-based application designed to centralize and streamline core academic operations. The system allows administrators to efficiently manage Departments, Lecturers, Courses, Students, and Course Enrollments through a unified interface.

The project strictly adheres to a **3-Tier Architecture** model to ensure separation of concerns, scalability, and security:

- **Tier 1: Presentation Layer:** Built using HTML, CSS (TailwindCSS), and JavaScript. It provides a responsive, modern user interface.
- **Tier 2: Application Layer:** Developed using Core PHP. It handles business logic, session-based authentication, form validation, and securely communicates with the database using Prepared Statements.
- **Tier 3: Data Layer:** Powered by MySQL. This layer goes beyond simple tables, implementing advanced relational database concepts including Stored Procedures, Triggers, Views, Transactions, and Indexes to maintain strict data integrity and automate business rules.



2. Entity Relationship (ER) Diagram



Entity Relationships:

- **Departments to Lecturers (1:M):** One department can have many lecturers, but a lecturer belongs to one specific department.
- **Lecturers to Courses (1:M):** A lecturer can teach multiple courses, but a specific course is assigned to one primary lecturer.
- **Students to Courses (M:M):** A student can enroll in multiple courses, and a course can have multiple students. This Many-to-Many relationship is resolved using the associative entity **Enrollments**.
- **System Users:** An independent entity used exclusively for handling administrative authentication and role-based access.

---

3. Database Normalization Process

Normalization is a systematic approach used in the UOVT Student Management System to organize data, reduce redundancy, and eliminate undesirable characteristics like Insertion, Update, and Deletion anomalies. Our database was designed by progressing through Unnormalized Form (UNF) to Third Normal Form (3NF).

Below is the step-by-step breakdown of how the raw data structures were refined into our final relational schema.

3.1 Unnormalized Form (UNF)

Initially, data requirements for the student management system can be visualized as a flat file or spreadsheet. This structure contained repeating groups and multi-valued attributes, making it highly inefficient for a relational database.

Table: UNF_StudentData

student_id	first_name	last_name	contact_email	Course1_ID	Course1_Name	Lect1_Name	Dept1_Name	Course2_ID	Course2_Name	Lect2_Name	Dept2_Name
1	Darsha	Anuradha	darsha@email.com	101	Database	Dr. Perera	IT	102	Python	Mr. Silva	IT

Issues Identified:

- Multiple courses per student create repeating groups (Course 1, Course 2, etc.).

- Data is not atomic.
- Extremely difficult to query, update, or scale.

3.2 First Normal Form (1NF)

To achieve 1NF, we eliminated repeating groups and ensured that every attribute contains only atomic (indivisible) values. We achieved this by creating a separate record for each course a student takes, resulting in a composite key scenario.

Table: 1NF_StudentRecords

student_id (PK)	course_id (PK)	first_name	last_name	contact_email	course_name	lecturer_id	lecturer_name	dept_id	dept_name
1	101	Darsha	Anuradha	darsha@email.com	Database	50	Dr. Perera	5	IT
1	102	Darsha	Anuradha	darsha@email.com	Python	51	Mr. Silva	5	IT

- **Actions Taken:** Repeating columns were removed. A composite Primary Key (student_id + course_id) conceptually identifies each row uniquely.
- **Remaining Issues:** Partial dependencies exist. first_name, last_name, and contact_email only depend on student_id. course_name, lecturer_name, and dept_name only depend on course_id.

3.3 Second Normal Form (2NF)

To achieve 2NF, the schema must be in 1NF, and all partial dependencies must be removed. Every non-key attribute must depend on the entire primary key. We resolved this by splitting the data into separate entities and creating a junction table for the many-to-many relationship.

Table 1: students

student_id (PK)	first_name	last_name	contact_email

Table 2: courses_2NF

course_id (PK)	course_name	lecturer_id	lecturer_name	dept_id	dept_name

Table 3: enrollments (Junction Table)

enrollment_id (PK)	student_id (FK)	course_id (FK)	enrollment_date

- **Actions Taken:** Created the enrollments table with its own surrogate primary key (enrollment_id) to track when students join courses. Separated courses and students into their own distinct tables.
- **Remaining Issues:** Transitive dependencies exist within the courses_2NF table. lecturer_name depends on lecturer_id, and dept_name depends on dept_id, rather than depending directly on the primary key (course_id).

3.4 Third Normal Form (3NF) - Final Schema

To achieve 3NF, the schema must be in 2NF, and all transitive dependencies must be removed. Non-key attributes must depend only on the primary key. We resolved the transitive dependencies by extracting the Lecturer and Department data into their own dedicated tables, linked via Foreign Keys.

Final 3NF Tables Implemented:

- **departments:** dept_id (PK), dept_name
- **lecturers:** lecturer_id (PK), name, email, dept_id (FK)
- **courses:** course_id (PK), course_name, lecturer_id (FK), total_enrolled
- **students:** student_id (PK), first_name, last_name, contact_email
- **enrollments:** enrollment_id (PK), student_id (FK), course_id (FK), enrollment_date
- **system_users:** user_id (PK), email, password_hash, role

Final Result: All tables have proper Primary Keys (PK) with AUTO_INCREMENT. Foreign Keys (FK) are established to maintain strict referential integrity. The database is fully normalized to 3NF, ensuring efficient CRUD operations via the PHP application layer without risk of data anomalies.

---

4. List of MySQL Features Implemented

Our Data Layer heavily utilizes advanced MySQL features to ensure security, performance, and automation.

4.1 Relational Tables & Constraints

- Created 6 normalized tables (departments, lecturers, courses, students, enrollments, system_users).
- Enforced Primary Keys for unique identification and Foreign Keys with ON DELETE CASCADE and ON DELETE SET NULL to maintain referential integrity.
- Utilized UNIQUE constraints to prevent duplicate emails and duplicate course enrollments.

4.2 Stored Procedures (14+ Implemented)

We encapsulated all major CRUD operations into stored procedures. This reduces network traffic, centralizes logic, and provides a strong defense against SQL injection.

- sp_get_courses, sp_get_course_by_id, sp_insert_course, sp_update_course, sp_delete_course
- sp_get_students, sp_insert_student, sp_update_student, sp_delete_student
- AddDepartment, UpdateDepartment, DeleteDepartment
- insert_lecturer, update_lecturer, delete_lecturer
- **EnrollStudent**: Handles the complex logic of enrolling a student.

4.3 Database Transactions

- **Feature:** Implemented inside the EnrollStudent stored procedure.
- **Explanation:** When a student is enrolled, two things must happen: a record is inserted into enrollments, and the total_enrolled counter in the courses table must be incremented. We wrapped these in a START TRANSACTION and COMMIT. If either fails, a ROLLBACK is triggered, ensuring the database is never left in a mismatched state.

4.4 Triggers

- **Feature:** After_Enrollment_Delete
- **Explanation:** When an administrator unenrolls a student (or a student is deleted, triggering a cascade delete in enrollments), this trigger automatically fires AFTER DELETE ON enrollments to decrement the total_enrolled count in the courses table. This automates business logic directly at the database level.

4.5 Database Views

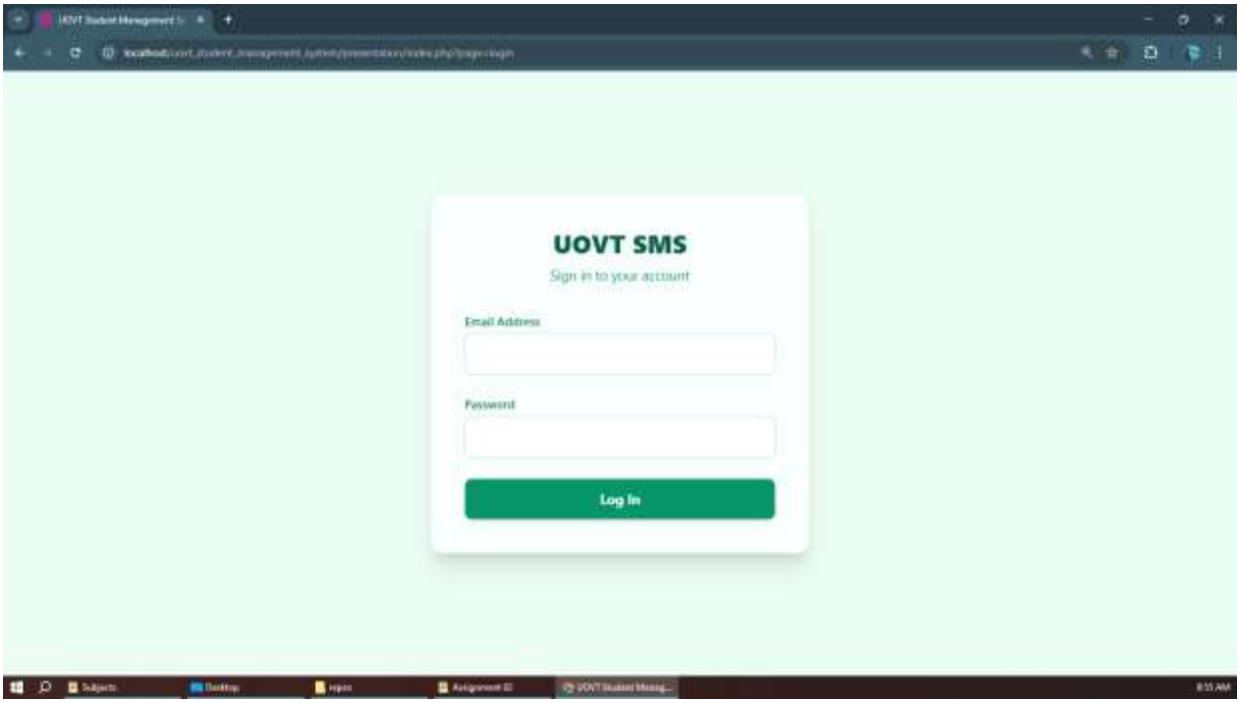
- **Feature:** ViewEnrollments, view_departments, lecturer_departments_view
- **Explanation:** Views were created to simplify complex SQL JOIN statements for the application layer. For example, ViewEnrollments securely combines data from enrollments, students, and courses into a single, easily readable virtual table used to generate the UI reports.

4.6 Indexes

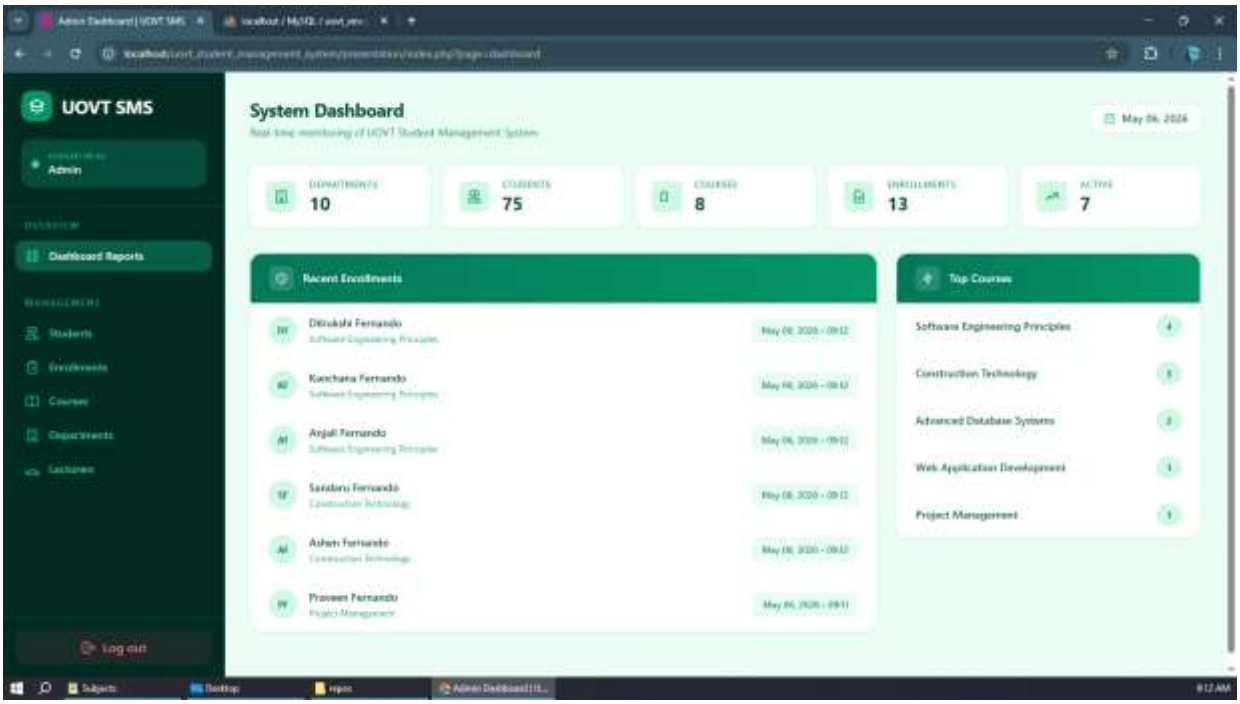
- **Feature:** idx_student_full_name, idx_student_email
- **Explanation:** Created on frequently searched/sorted columns to optimize query read performance as the database grows.

5. Application Screenshots

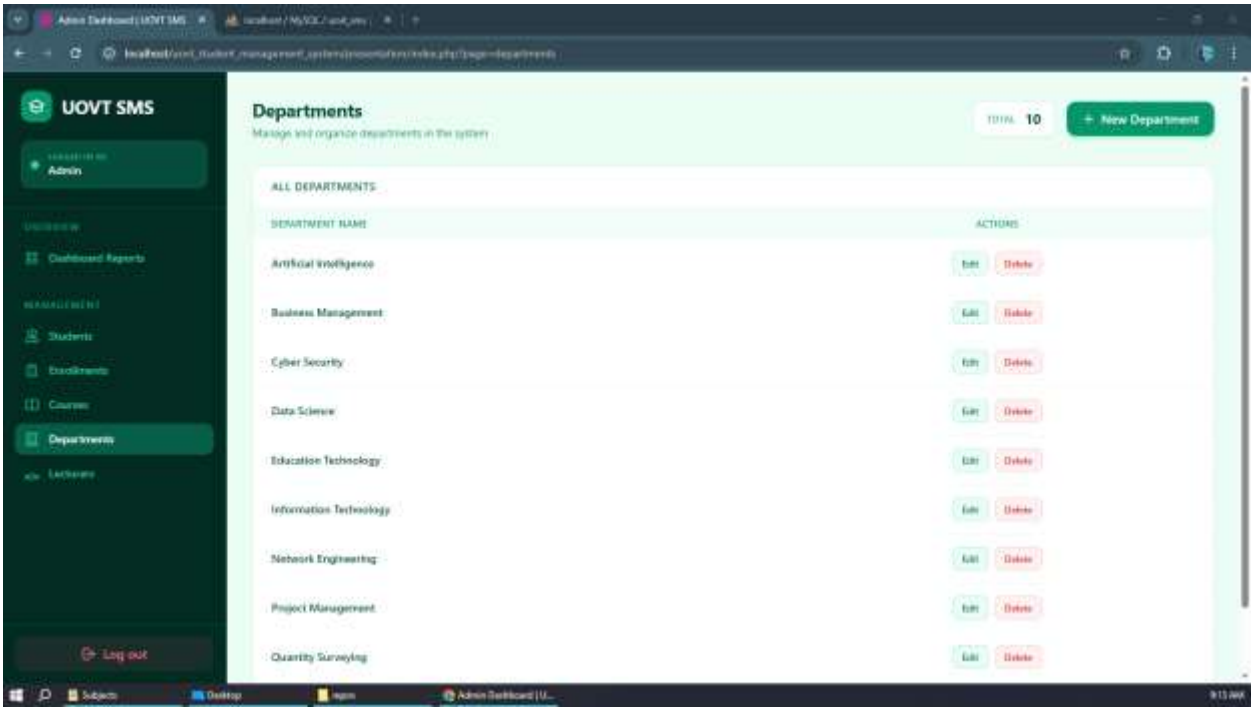
1. **Secure Login Page:**



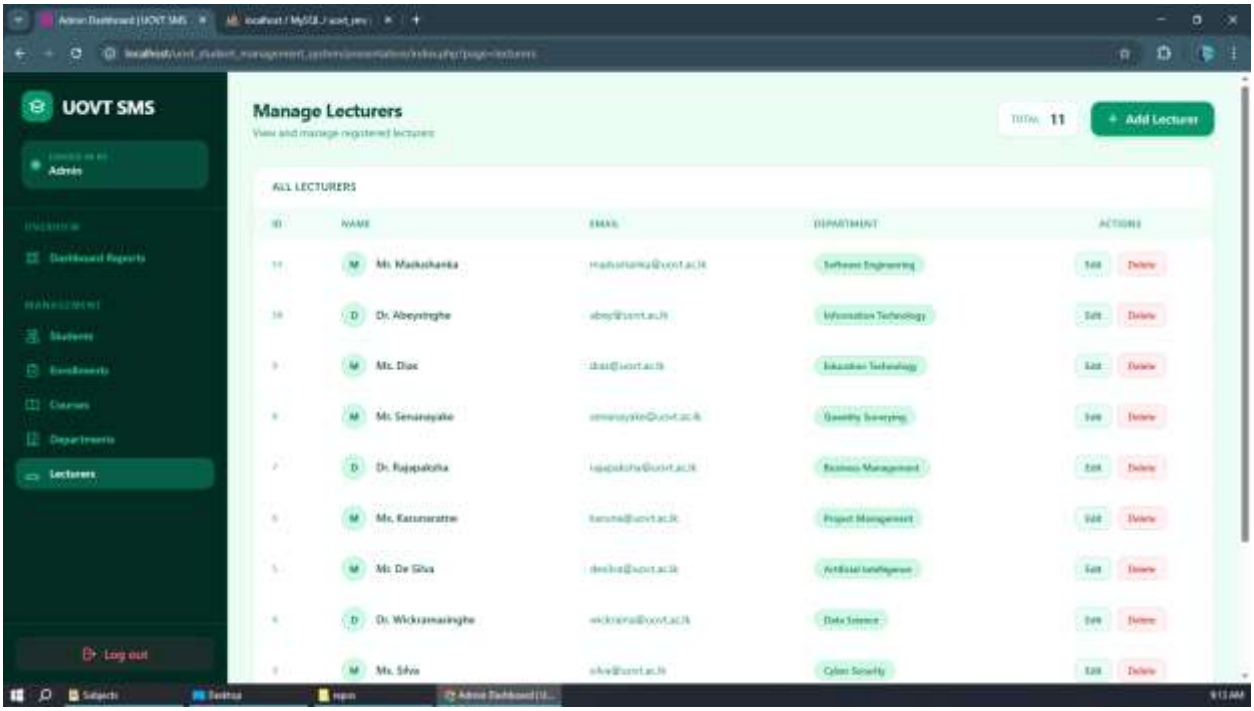
2. **System Dashboard (Overview):**



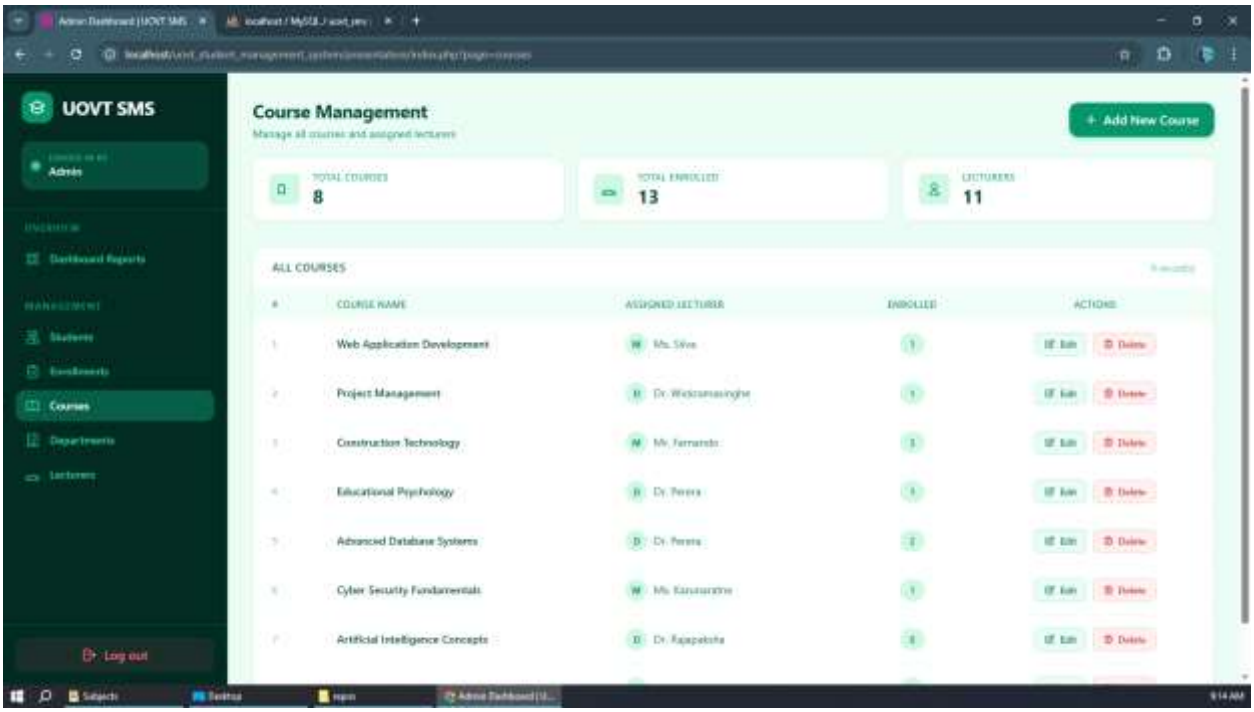
3. **Manage Departments:**



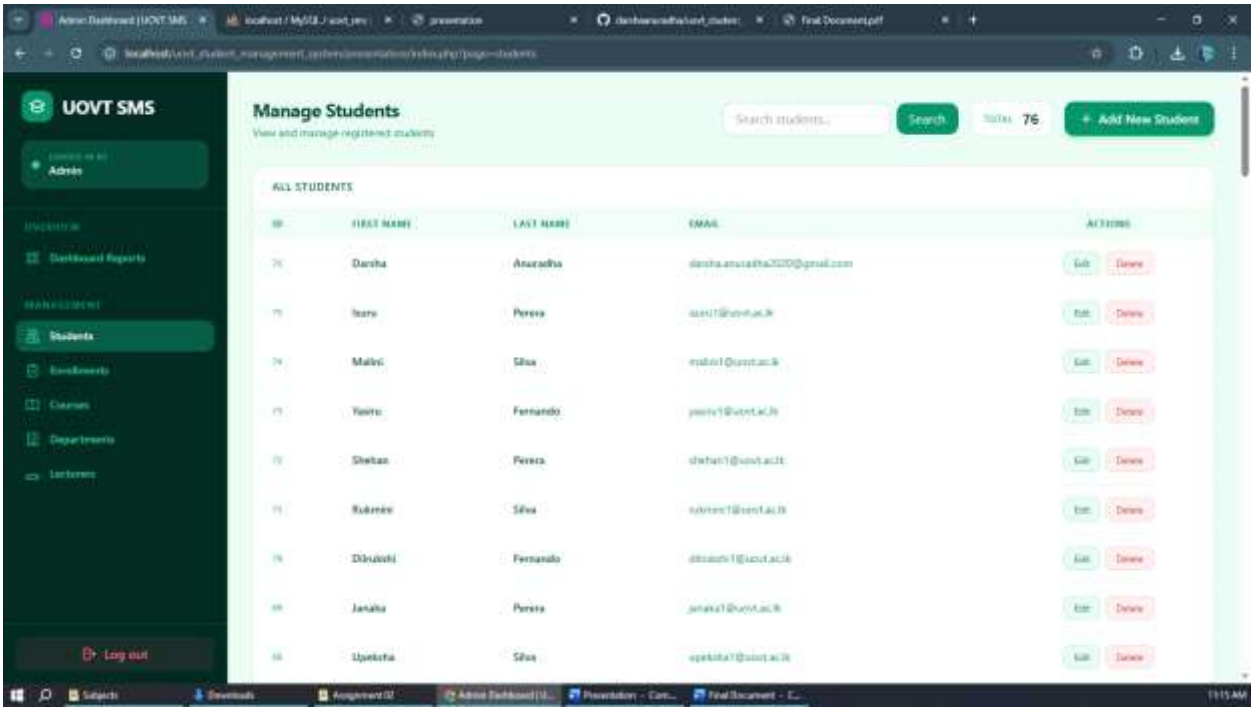
4. **Manage Lecturers:**



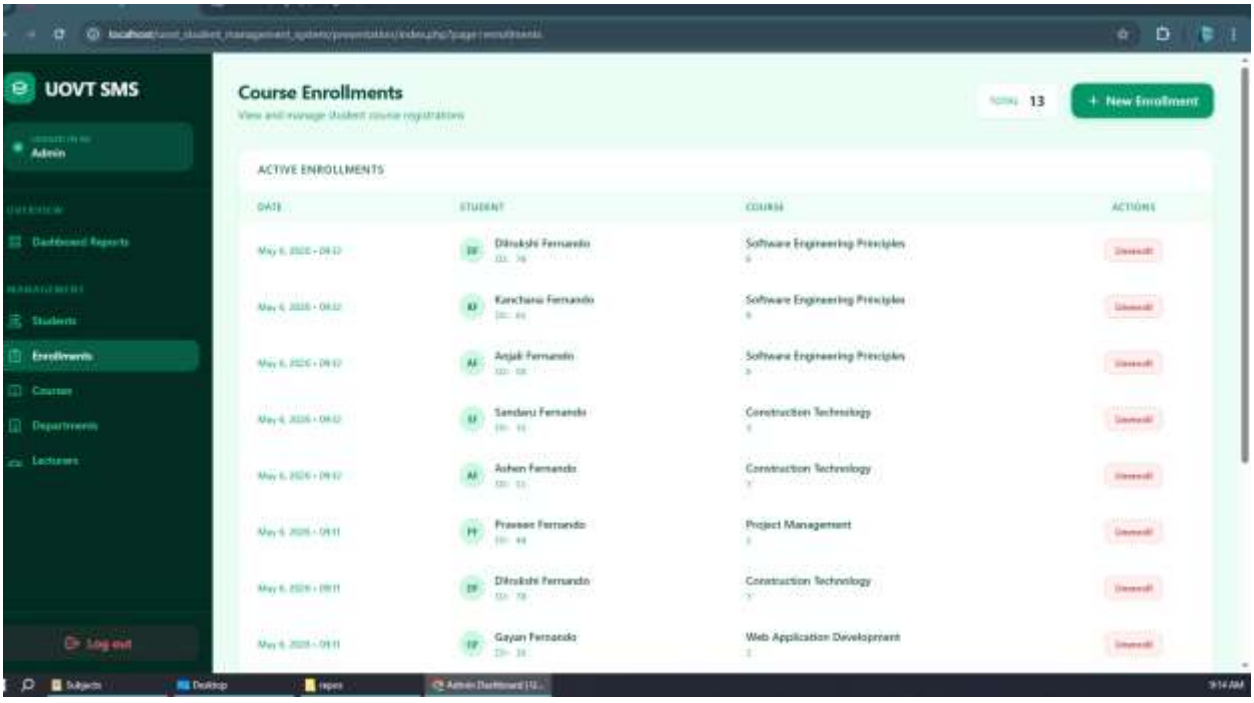
5. **Manage Courses:**

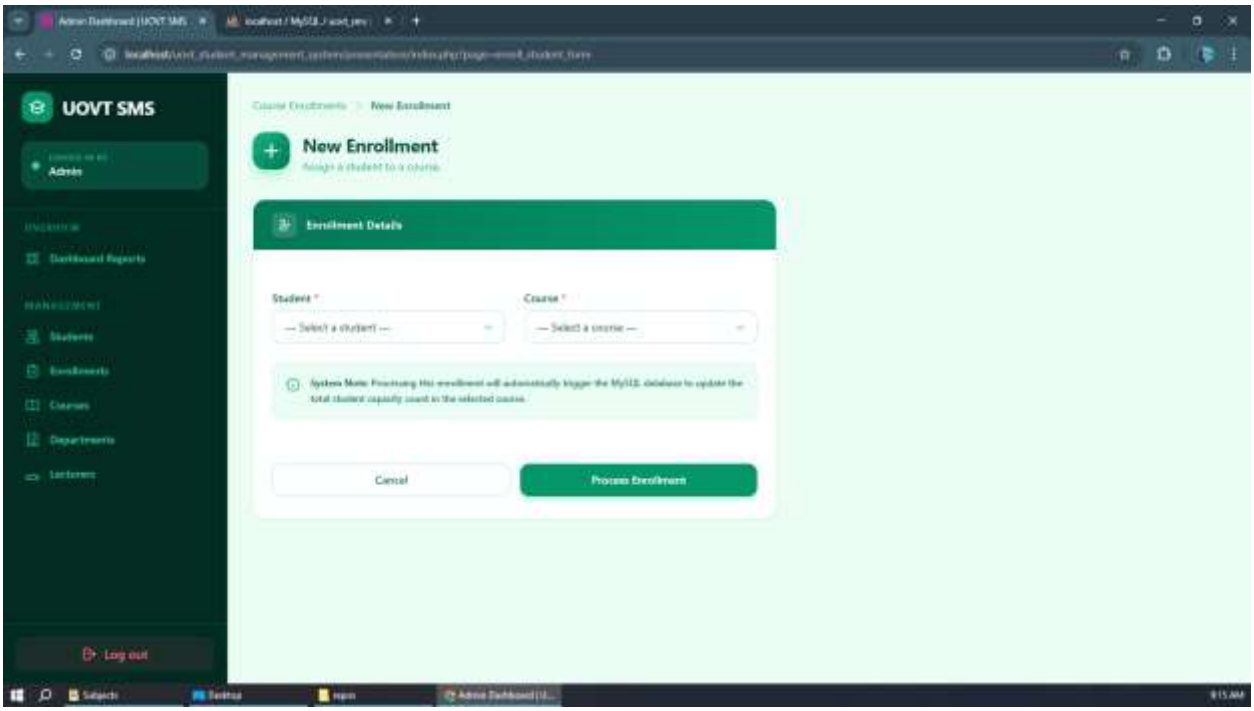


6. **Manage Students:**



7. **Course Enrollments Workflow:**



8. 

6. Team Contribution Table

As required by the assignment guidelines, the workload was evenly distributed among the team members. Every member contributed to the database design, SQL logic, and PHP backend for their respective modules.

Member Name	Student ID	Specific Contributions
G. B. D. Anuradha	SIS24B215	Developed the System Dashboard, Enrollments module, Schema, ER, associated final documentation
L. B. Charith Jeewan	SIS24B236	Developed the Departments module, Stored Procedures, Documented the Database Normalization process.
W. I. L. Withana	SIS24B238	Developed the Students module (CRUD operations), associated Stored Procedures, UI integration.
H. K. G. V. L. Koralage	SIS24B213	Developed the Lecturers module, Views, associated Stored Procedures, associated final documentation
B. W. S. S. Nawarathna	SIS24B239	Developed the Courses module, associated Stored Procedures. UI integration.

---

7. Challenges Faced and Solutions

During the development of the 3-Tier application, the team encountered and overcame several technical challenges:

Challenge 1: Maintaining Data Consistency for Enrollments

- **Issue:** We needed a way to track the total number of students in a course. Doing this with PHP logic was risky; if the PHP script failed halfway, the enrollment count would be inaccurate.
- **Solution:** We shifted this responsibility to the Data Layer. We used a MySQL Transaction inside the EnrollStudent stored procedure to handle increments, and a MySQL Trigger (After_Enrollment_Delete) to handle decrements. This guaranteed mathematical accuracy at the database level regardless of the Application Layer.

Challenge 2: Preventing SQL Injection

- **Issue:** Direct insertion of form data into SQL queries left the application vulnerable to injection attacks.
- **Solution:** We strictly enforced the use of MySQLi Prepared Statements across all PHP files. Furthermore, because we used Stored Procedures for all major queries, user inputs were treated strictly as parameters, neutralizing injection risks.

Challenge 3: Complex Joins Cluttering PHP Code

- **Issue:** The presentation layer required data from multiple tables (e.g., showing a Lecturer's name alongside a Course, or a Department name alongside a Lecturer). Writing complex JOIN queries inside PHP made the Application Layer messy.
- **Solution:** We created MySQL Views (such as lecturer_departments_view and ViewEnrollments). This allowed our PHP code to run simple SELECT * FROM View statements, keeping the Application Layer clean and relying on the Database Layer to process the relational logic.

Challenge 4: UI/UX Consistency

- **Issue:** Because five different team members were working on five different modules, the initial user interfaces looked mismatched.

- **Solution:** We established a centralized "Flat Emerald" design system using TailwindCSS. We created a master layout and shared UI components to ensure the Presentation Layer remained highly professional and consistent across all pages.